



TP Scraping – Kiprix

Systeme de Scraping Orienté Objet



Système de Scraping Orienté Objet

Travail réalisé par :

SADI Mohand

GUINDO Ibrahim Ousseini

AKONOU Gbemahonmede

I. Introduction

Dans le cadre du module de Programmation Orientée Objet (POO) en L2 Informatique, nous avons réalisé un projet de scraping web appliqué au site .

L'objectif principal de ce projet est de concevoir un système logiciel modulaire capable d'extraire, structurer et analyser des données relatives aux écarts de prix entre la France métropolitaine et les territoires d'outre-mer (DOM).

Au-delà du simple scraping, ce projet vise à mettre en pratique plusieurs concepts fondamentaux étudiés en cours :

- Conception orientée objet (abstraction, héritage, modularité)
- Architecture logicielle propre et maintenable
- Tests unitaires avec couverture de code
- Gestion des erreurs réseau et respect des règles éthiques
- Containerisation avec Docker pour assurer la portabilité

Le système développé repose sur une architecture organisée en plusieurs modules distincts : un scraper générique, un scraper spécialisé pour Kiprix, un gestionnaire central, un système de cache, un module d'analyse de données ainsi qu'une interface en ligne de commande.

Ce rapport présente l'architecture du projet, les choix techniques réalisés, la mise en place des tests et les aspects DevOps associés.

II. Architecture du Projet

L'architecture du système a été conçue selon une approche modulaire et orientée objet afin d'assurer la maintenabilité, l'évolutivité et la clarté du code.

Le projet est structuré autour du dossier `src/`, qui contient les différents modules responsables de la logique métier.

1. **BaseScraper:**

La classe BaseScraper constitue le cœur du système.

Il s'agit d'une classe abstraite définissant :

- La récupération des pages web (`fetch_page`)
- La gestion des erreurs HTTP et réseau
- Le respect d'un délai entre les requêtes
- Les méthodes de sauvegarde en JSON et CSV

Cette abstraction permet de garantir une structure commune à tous les scrapers spécialisés.

2. **KiprixScraper**

La classe KiprixScraper hérite de BaseScraper.

Elle implémente :

- La logique de pagination
- L'extraction des produits
- L'analyse des champs spécifiques (prix, différence, territoire)

Cette spécialisation illustre le principe d'héritage en programmation orientée objet.

3. **ScraperManager**

Le module ScraperManager joue le rôle de Factory.

Il permet :

- La création dynamique des scrapers
- La gestion centralisée des différents types de scraping

Ce choix améliore la modularité du système.

4. **CacheManager**

Le système CacheManager permet d'éviter les requêtes redondantes en stockant temporairement les pages récupérées.

Avantages :

- Réduction du trafic réseau
- Respect des bonnes pratiques éthiques
- Amélioration des performances

5. **DataAnalyzer**

Le module DataAnalyzer exploite la bibliothèque pandas pour :

- Charger les données
- Produire des statistiques descriptives
- Analyser les écarts de prix
- Exporter les résultats

6. **Interface CLI**

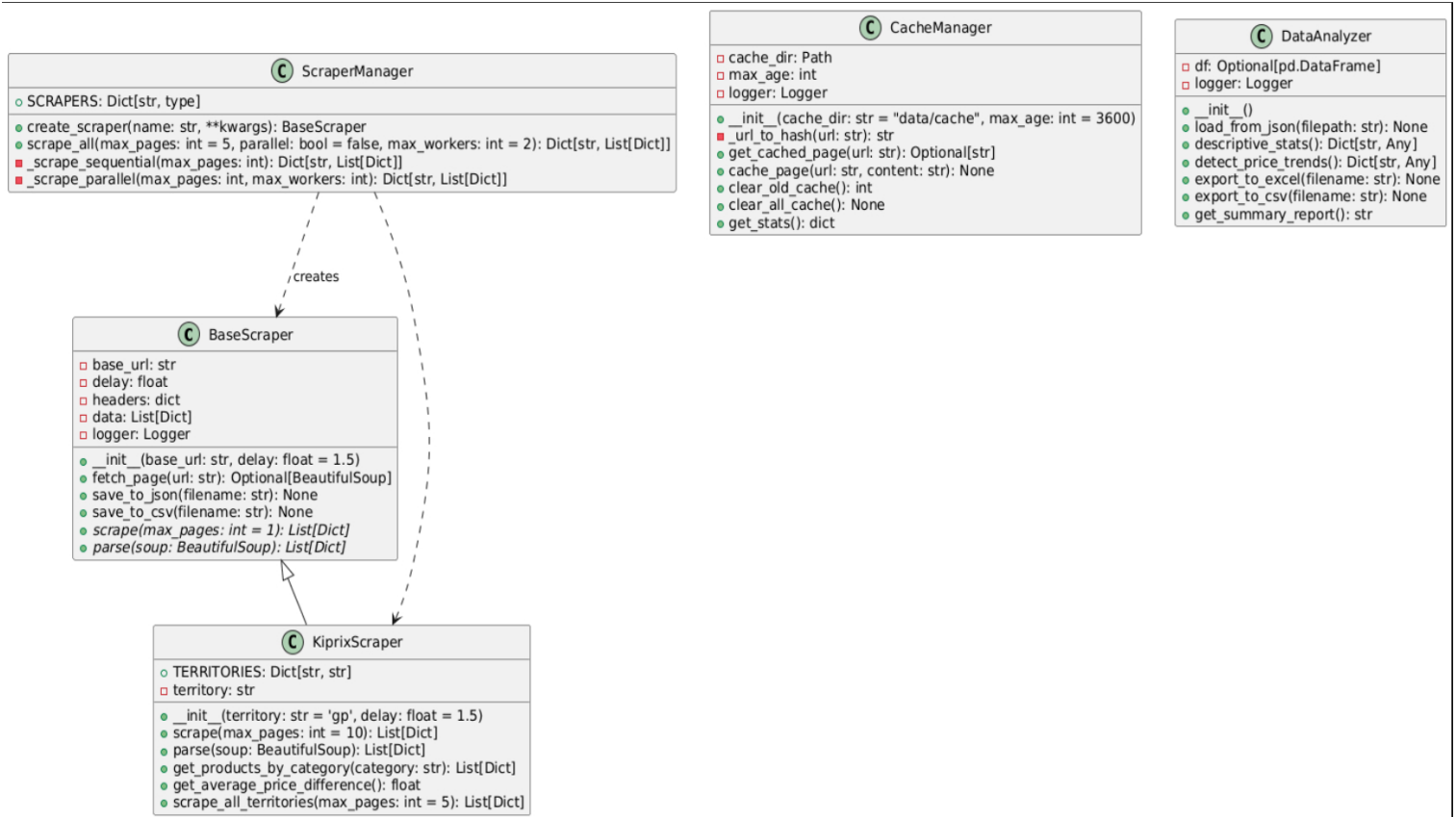
L'interface en ligne de commande (CLI) permet :

- De lancer le scraping
- D'analyser les données
- De gérer le cache

Elle rend le projet utilisable comme un véritable outil logiciel.

III. **Diagramme UML**

Le diagramme UML ci-dessous représente l'architecture orientée objet du projet, en mettant en évidence les relations d'héritage et de dépendance entre les différentes classes.



1. Relation d'héritage :

La classe **KiprixScraper** hérite de **BaseScraper**.

Cette relation illustre le principe d'héritage, permettant de réutiliser :

- La gestion des requêtes HTTP
- Le traitement des erreurs
- Les méthodes de sauvegarde
- Le système de délai entre requêtes

Ainsi, toute nouvelle implémentation de scraper pourra étendre BaseScraper sans modifier son fonctionnement interne.

2. Relations de dépendance :

Plusieurs relations de dépendance sont visibles dans le diagramme :

- **ScraperManager** est responsable de la création et de la gestion des scrapers.
- Il instancie dynamiquement les classes dérivées de **BaseScraper**.
- **CacheManager** fonctionne comme un module indépendant dédié à l'optimisation des requêtes.
- **DataAnalyzer** constitue un module séparé chargé de l'analyse statistique des données extraites.

Cette organisation reflète une séparation claire des responsabilités entre les composants du système.

3. Principes de conception :

L'architecture respecte plusieurs principes de conception logicielle :

- **Single Responsibility Principle** : chaque classe possède un rôle bien défini.
- **Open/Closed Principle** : il est possible d'ajouter de nouveaux scrapers en étendant **BaseScraper** sans modifier l'architecture existante.
- **Modularité** : les modules de scraping, de cache et d'analyse sont distincts et indépendants.

Cette conception rend le système maintenable, évolutif et structuré.

IV. Tests et Qualité Logicielle

La qualité du logiciel a constitué un axe central du projet.

Des tests unitaires ont été développés à l'aide du framework **pytest** afin de garantir la robustesse, la fiabilité et la maintenabilité du système.

1. Objectif de couverture :

L'objectif fixé était d'atteindre un taux de couverture supérieur à 70%.

Après implémentation des tests, la couverture globale atteint : 72%

Ce résultat valide la conformité aux exigences du projet.

Coverage report: 72%

[Files](#)[Functions](#)[Classes](#)

coverage.py v7.13.4, created at 2026-03-01 17:30 -0400

File ▲	class	statements	missing	excluded	coverage
src__init__.py	(no class)	0	0	0	100%
src\analyzer.py	DataAnalyzer	31	5	0	84%
src\analyzer.py	(no class)	14	0	0	100%
src\base_scraper.py	BaseScraper	45	3	0	93%
src\base_scraper.py	(no class)	19	0	0	100%
src\cache.py	CacheManager	35	8	0	77%
src\cache.py	(no class)	14	0	0	100%
src\cli.py	(no class)	62	62	0	0%
src\manager.py	ScraperManager	26	3	0	88%
src\manager.py	(no class)	10	0	0	100%
src\scrapers__init__.py	(no class)	2	0	0	100%
src\scrapers\kiprix_scraper.py	KiprixScraper	43	6	0	86%
src\scrapers\kiprix_scraper.py	(no class)	12	0	0	100%
Total		313	87	0	72%

2. Stratégie de test:

Les tests couvrent les modules principaux du dossier `src/` :

- BaseScraper
- KiprixScraper
- ScraperManager
- CacheManager
- DataAnalyzer

Les scénarios testés incluent :

- Initialisation des classes
- Gestion des erreurs HTTP (404, Timeout, ConnectionError)
- Fonctionnement du système de cache
- Sauvegarde des données (JSON et CSV)
- Analyse statistique
- Exécution séquentielle et parallèle du scraping

3. Techniques utilisées:

Plusieurs techniques avancées ont été employées :

- Mocking des requêtes HTTP avec `unittest.mock`
- Simulation d'erreurs réseau
- Utilisation de `monkeypatch` pour rediriger les chemins de fichiers
- Utilisation de `tmp_path` pour les fichiers temporaires
- Tests sur les exceptions et comportements limites

Ces pratiques permettent de tester le comportement du système sans effectuer de véritables requêtes réseau.

4. Robustesse du système

Les tests garantissent :

- La gestion correcte des erreurs réseau
- L'absence d'échec lors de données vides
- Le respect des comportements attendus

Cette approche assure une meilleure stabilité et réduit les risques de régression lors de futures évolutions.

V. DevOps et Containerisation:

Afin d'assurer la portabilité et la reproductibilité de l'environnement d'exécution, le projet a été containerisé à l'aide de **Docker**.

La containerisation permet d'exécuter le scraper dans un environnement isolé, indépendamment du système d'exploitation hôte.

1. Objectifs de la containerisation :

L'utilisation de Docker permet :

- D'éviter les problèmes de dépendances
- De garantir un environnement Python cohérent
- De faciliter le déploiement sur n'importe quelle machine
- De rendre le projet facilement partageable

2. Construction de l'image:

Le fichier `Dockerfile` définit :

- L'image de base (Python 3.11 slim)
- L'installation des dépendances via `requirements.txt`
- La copie du code source
- La commande d'exécution

Commande utilisée :

✓ `docker build -t kiprix-app .`

3. Exécution du container:

Le scraping peut être lancé via :

✓ `docker run kiprix-app python scrape --pages 1 --territory gp`

Cette commande exécute le programme dans un environnement isolé et produit les fichiers de sortie dans le container.

4. Avantages obtenus:

La mise en place de Docker améliore :

- ✓ La robustesse du projet
- ✓ La portabilité
- ✓ La facilité d'évaluation par le professeur

Le projet peut ainsi être exécuté sans configuration supplémentaire.

VI. Respect des règles éthiques

Le scraping web implique des responsabilités techniques et éthiques.

Dans ce projet, plusieurs bonnes pratiques ont été mises en place afin de respecter les serveurs distants et limiter l'impact des requêtes.

1. **User-Agent explicite:**

Les requêtes HTTP incluent un User-Agent personnalisé, permettant d'identifier clairement le type d'application effectuant la requête.

Cela améliore la transparence vis-à-vis du serveur cible.

2. **Délai entre les requêtes (Rate Limiting):**

Un délai configurable est appliqué entre chaque requête HTTP.

Cette stratégie permet :

- De réduire la charge sur le serveur
- D'éviter un comportement assimilable à une attaque
- De respecter les bonnes pratiques du scraping responsable

3. **Gestion des erreurs réseau:**

Le système gère correctement :

- Les erreurs HTTP (404, 500)
- Les timeouts
- Les erreurs de connexion

Cela évite les comportements imprévus et améliore la stabilité globale du système.

4. **Système de cache:**

L'utilisation d'un CacheManager permet d'éviter les requêtes répétées vers les mêmes pages. Cette optimisation réduit la charge réseau et améliore les performances.

✓  Le projet adopte ainsi une approche respectueuse et responsable du scraping web.

VII. Conclusion

Ce projet de scraping appliqué au site nous a permis de mettre en pratique de manière concrète les principes fondamentaux de la programmation orientée objet étudiés en cours.

Au-delà de l'extraction de données, nous avons conçu une architecture modulaire et maintenable reposant sur l'abstraction, l'héritage et la séparation des responsabilités.

La mise en place de tests unitaires avec un objectif de couverture supérieur à 70% a renforcé la robustesse du système, tandis que l'intégration de Docker a permis d'assurer la portabilité et la reproductibilité de l'environnement d'exécution.

Ce projet illustre une approche complète de développement logiciel intégrant :

- Conception architecturale
- Qualité logicielle
- Analyse de données
- Bonnes pratiques DevOps
- Respect des règles éthiques du scraping

Il constitue une base solide pour des projets plus avancés en ingénierie logicielle et en data engineering.

Enfin, ce travail collaboratif nous a permis de développer des compétences techniques mais également organisationnelles, notamment dans la répartition des rôles et la coordination du développement.

Ainsi, ce projet représente une expérience formatrice combinant rigueur technique et approche professionnelle du développement logiciel.